

20th September 2024



Advanced XSLT: Running XSLT from API

O'Neil Delpratt

Saxonica

www.saxonica.com

1. Implementations of XSLT 3.0
2. Different ways to run XSLT
3. Running XSLT from API



LANDSCAPE OF XSLT 3.0

- Saxon – XSLT 3.0, XQuery 3.1, XPath 3.0/3.1, XML Schema
- Altova
- Xalan-J – streaming not supported

Running XSLT 3.0 Stylesheets

- Command Line
- Saxon - Gizmo
- Oxygen, Stylus Studio
- Saxon API
 - Java, C#, C/C++, Python, PHP
 - SaxonJS: JavaScript in-browser support

Running XSLT 3.0 Stylesheets

Github project needed for exercises:

```
git clone https://github.com/ond1/xmlss  
cd xmlss
```

Running XSLT Stylesheets from command-line

```
java -cp saxon-he-12.5.jar net.sf.saxon.Transform -s:xml/books.xml  
-xsl:stylesheets/books.xsl
```

Output of the primary document to stdout:

```
<!DOCTYPE HTML>  
<html><!--Generated at 2024-09-16T17:59:48.820592+01:00--><!--some book text  
is here-->  
  <head>  
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">  
    <title>Book List</title>  
  </head>  
  <body leftmargin="100">  
    <h2>This week's top author is Jasper Fforde</h2>  
    <p>We stock the following 2 books by this author:</p>  
    <ul>  
      <li>The Big Over Easy</li>  
      <li>The Eyre Affair</li>  
    </ul>
```

Stylesheet example: books.xsl

Books.xsl:

```
<xsl:transform
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  version="3.0"
  expand-text="yes">
```

```
<!-- This style sheet displays the books.xml file. -->
```

```
<xsl:key name="authkey" match="ITEM" use="AUTHOR"/>
<xsl:key name="codekey" match="CATEGORY" use="@CODE"/>
```

```
<xsl:variable name="categories" select="//CATEGORY"/>
```

```
<xsl:variable name="now" select="current-dateTime()"/>
```

```
<xsl:param name="top-author">Jasper Fforde</xsl:param>
```

```
<xsl:param name="title">Book List</xsl:param>
```

```
....
```

Use of keys

Use of
global
parameters

Setting stylesheet parameters:

`param-name=value`

```
java -cp saxon-he-12.5.jar net.sf.saxon.Transform -s:books.xml -xsl:books.xsl  
top-author="Jane Austen"
```

Notice the content on the output changes

<https://www.saxonica.com/documentation12/index.html#!using-xsl/commandline>

Running XSLT Stylesheets from command-line

Saving primary document output in a file

```
java -cp saxon-he-12.5.jar net.sf.saxon.Transform -s:books.xml -xsl:books.xsl  
-o:books.html
```

<https://www.saxonica.com/documentation12/index.html#!using-xsl/commandline>

Gizmo:

- Command line utility,
- Operate interactively or in batch mode
- Perform simple operations such as examining the content of a document, renaming or deletion of selected elements in the document,
- Performing simple XSLT transformation and XSD validation.

```
java -cp .:saxon-he-12.5.jar net.sf.saxon.Gizmo -s:book.xml
```

Command-line tool: The Gizmo utility

```
/>?
```

Commands available:

```
call {filename} -- execute script from a file
copy {expression} -- make deep copy of all selected nodes
delete {expression} -- delete all selected nodes, with their content
follow {expression} with {query} -- add result of query after each selected node
help {keyword} -- help on a specific command, or '?' for all commands
list {expression} -- display paths of selected nodes
load {fileName} -- load new source document from file
namespace {prefix} {uri} -- bind namespace prefix to URI
paths -- display all distinct element paths in the document
precede {expression} with {query} -- add result of query before each selected node
prefix {expression} with {query} -- add result of query as first child of each selected node
quit [now] -- stop Gizmo
rename {expression-1} as {expression-2} -- change the name of selected nodes
replace {expression} with {query} -- replace selected nodes with result of query
save {filename} {output-property=value}... -- save current document to file
schema {filename} -- load XSD 1.1 schema for use in validation
set {variable} {expression} -- set variable to value of expression
show {expression} -- display content of all selected nodes
strip -- delete whitespace text nodes
suffix {expression} with {query} -- add result of query as last child of each selected node
transform {filename} -- transform current document using stylesheet in named file
undo -- revert the most recent changes
update {expression} with {query} -- replace content of selected nodes with result of query
validate -- validate against loaded schema and/or xsi:schemaLocation
```

Command-line tool: The Gizmo utility

```
/>show //BOOKLIST
```

```
<BOOKLIST>
```

```
<BOOKS>
```

```
<ITEM CAT="MMP">
```

```
<TITLE>Pride and Prejudice</TITLE>
```

```
<AUTHOR>Jane Austen</AUTHOR>
```

```
<PUBLISHER>Modern Library</PUBLISHER>
```

```
<PUB-DATE>2002-12-31</PUB-DATE>
```

```
<LANGUAGE>English</LANGUAGE>
```

```
<PRICE>9</PRICE>
```

```
<QUANTITY>187</QUANTITY>
```

```
<ISBN>0679601686</ISBN>
```

```
<PAGES>352</PAGES>
```

```
<DIMENSIONS UNIT="in">8.3 5.7 1.1</DIMENSIONS>
```

```
<WEIGHT UNIT="oz">6.1</WEIGHT>
```

```
</ITEM>
```

```
<ITEM CAT="P">
```

```
.....
```

Other operations:
transform {filename}



RUNNING XSLT FROM SAXON API

Available languages: Java, C/C++, PHP, **Python**, C#, JavaScript

1. Compiling a stylesheet
2. Setup source documents
3. Setup handling of results
4. Execute the compiled stylesheet

Focus will be on XSLT in Python

SaxonC Python

1. Installation

- Prerequisite: Python installed.

[Documentation](#)

- `pip install saxonche` or
use PyCharm IDE

PySaxonProcessor

Factory class for generating XQuery, XPath, Schema and XSLT compilers.

[Documentation](#)

```
proc = new PySaxonProcessor()  
xslt = proc.new_xslt30_processor()
```

PySaxonProcessor

For advanced features such as performance optimizations and Schema validation you will need `saxoncpe` or `saxoncee`

[Documentation](#)

```
proc = new PySaxonProcessor(license = True)
Xslt_proc = proc.new_xslt30_processor()
```

PyXslt30Processor

Set any options that are local to a specific compilation of the XSLT stylesheet.

Stylesheet options: What are these?

```
proc = new PySaxonProcessor(license = False)
xslt_proc = proc.new_xslt30_processor()
```

PyXslt30Processor

Stylesheet options: What are these?

PyXslt30Processor

Stylesheet options: What are these?

- Setting global parameters
- Import XSLT packages

PyXsltExecutable

Represents the compiled form of a stylesheet.

```
proc = new PySaxonProcessor(license = True)
xslt_proc = proc.new_xslt30_processor()
author_value = proc.make_string_value('Jane Austen')
xslt_proc.set_parameter('top-author', author_value)
executable =
xslt_proc.compile_stylesheet(stylesheet_file='books.xsl')
```

Value (<a>, , <c>) (1, 2, 3)

Item <a>

Atomic Value 1

PyXsltExecutable

Represents the compiled form of a stylesheet.

```
proc = new PySaxonProcessor()  
xslt_proc = proc.new_xslt30_processor()  
author_value = proc.make_string_value('Jane Austen')  
xslt_proc.set_parameter('top-author', author_value)  
executable =  
xslt_proc.compile_stylesheet(stylesheet_file='books.xsl')
```

PyXsltExecutable

What's missing?

PyXsltExecutable

What's missing?

Using Source Document (i.e XML document or JSON)

We are applying the XSLT stylesheet against the source document.

PyXsltExecutable

Source documents need parsing or passing directly to the compiled stylesheet

```
proc = new PySaxonProcessor()  
xml_doc = proc.parse_xml(xml_file_name='books.xml')
```

PyXsltExecutable

Represents the compiled form of a stylesheet.

```
proc = new PySaxonProcessor()
xml_doc = proc.parse_xml(xml_file_name='books.xml')
xslt_proc = proc.new_xslt30_processor()
author_value = proc.make_string_value('Jane Austen')

xslt_proc.set_parameter('top-author', author_value)
executable = xslt_proc.compile_stylesheet(stylesheet_file='books.xsl')

executable.apply_templates_returning_file(xdm_value=xml_doc,
output_file='books.html')
```

Python try except block

```
try:
    proc = new PySaxonProcessor()
    xml_doc = proc.parse_xml(xml_file_name='books.xml')
    xslt_proc = proc.new_xslt30_processor()
    author_value = proc.make_string_value('Jane Austen')

    xslt_proc.set_parameter('top-author', author_value)
    executable = xslt_proc.compile_stylesheet(stylesheet_file='books.xsl')

    executable.apply_templates_returning_file(xdm_value=xml_doc, output_file='books.html')
except Exception as err:
    print(err)
```

Global context item not set

Error evaluating (root) in expression in `xsl:variable/@select` on line 11 column 56 of `books.xsl`:
XPDY0002 Finding root of tree: the context item is absent
In variable `categories` on line 11 column 56 of `books.xsl`:
invoked by global variable `categories` at `file:///Users/ond1/Downloads/saxonjs3-demo/build/website/books.xsl#11`
In template rule with `match="BOOKLIST"` on line 48 of `books.xsl`
invoked by `xsl:apply-templates` at `file:///Users/ond1/Downloads/saxonjs3-demo/build/website/books.xsl#29`
In template rule with `match="/"` on line 19 of `books.xsl`

What we need:

```
executable.set_global_context_item(xdm_item=xml_doc)
```

Because:

```
<xsl:variable name="categories" select="//CATEGORY"/>
```

PyXsltExecutable

Handling Output

`apply_templates_returning_string`

`apply_templates_returning_file`

`apply_templates_returning_value`

Secondary Result Document

JSON notation syntax:

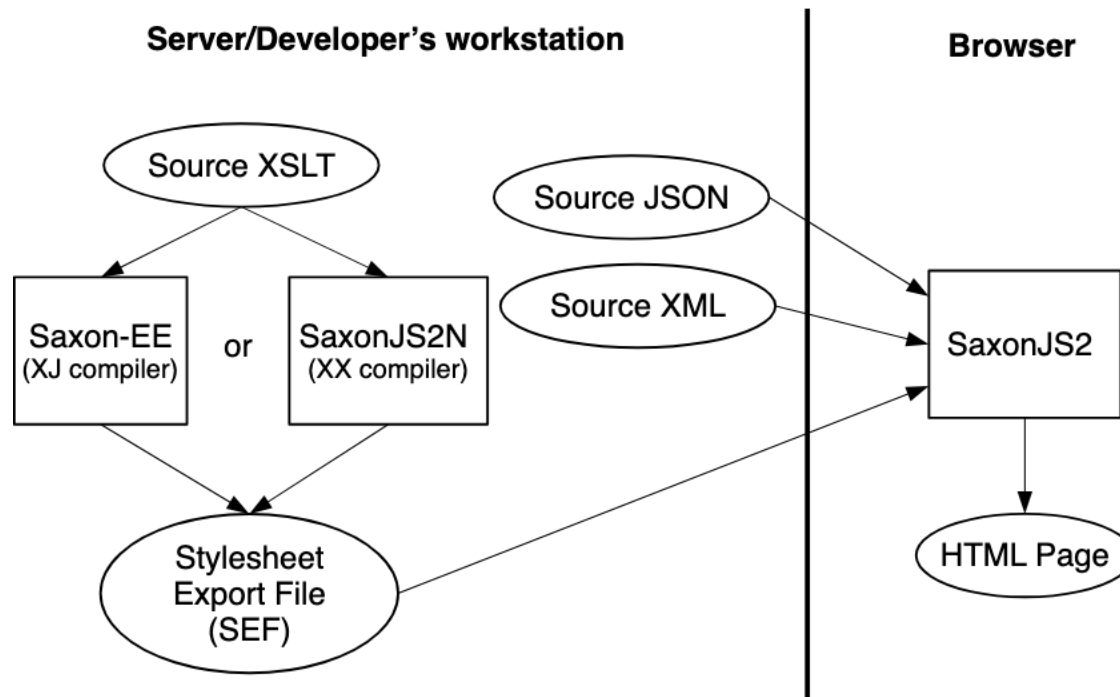
- Data is in name/value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays

Parsing JSON

JSON <-> PyXdmMap

```
json1 = """{ "test" : "This is a test. Price is higher than £25. " }"""  
parsed_json1 = saxonproc.parse_json(json_text=json1)
```

SaxonJS example



Any questions?

Thank you for your attention.

O'Neil Delpratt

<http://www.saxonica.com>

oneil@saxonica.com